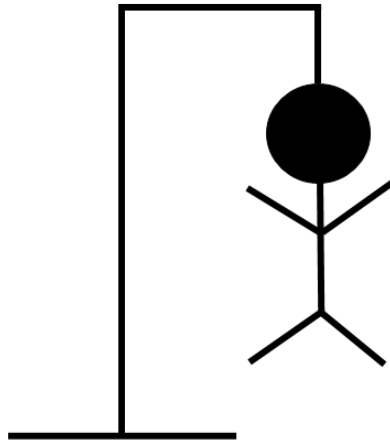


Manual Técnico del Juego Ahorcado

Juego Ahorcado



Hecho por Cynthia Vanessa Rodriguez Silva

Índice

Manual Técnico del Juego Ahorcado.....	1
Introducción	3
Arquitectura del Sistema.....	3
Estructura de Clases	3
Paradigmas Implementados	3
Sintaxis y Semántica	3
Administración de Memoria	4
Manejo de Errores	5
Programación Orientada a Objetos (POO)	6
Abstracción	6
Encapsulación	6
Modularidad.....	7
Herencia	7
Programación Orientada a Eventos.....	7
Flujo del Programa.....	8
Consideraciones Técnicas	8
Requisitos del Sistema	8
Link de la aplicación.....	8
Código de la Lógica del Juego.....	9
.....	9
Conclusiones	11

Introducción

Este manual técnico documenta en detalle el diseño, la arquitectura y la implementación del Juego del Ahorcado, desarrollado en C# utilizando Windows Forms. El objetivo principal es proporcionar una guía clara sobre la estructura del sistema, los paradigmas de programación aplicados y las buenas prácticas implementadas en el código.

El juego sigue las reglas tradicionales del ahorcado, donde el jugador debe adivinar una palabra oculta letra por letra, con un límite de intentos fallidos.

Arquitectura del Sistema

Estructura de Clases

Juego (Form principal): Maneja la interfaz gráfica y eventos del usuario.

LogicaJuego: Contiene toda la lógica del juego.

EstadoJuego: Modelo para representar el estado actual del juego.

ResultadoJuego: Modelo para representar resultados de operaciones.

MensajePersonalizado: Form para mostrar mensajes al usuario.

ConfirmarSalir: Form para confirmar salida del juego.

Paradigmas Implementados

Sintaxis y Semántica

El código sigue las reglas sintácticas de C#:

Nombres descriptivos para variables y métodos (`_palabraActual`, `ProcesarIntento`).

Uso de tipos de datos (`string`, `int`, `bool`).

Estructuras de control (`if-else`, bucles).

Comentarios explicativos donde es necesario.

Ejemplo de validación semántica:

```
private bool ValidarLetra(string letra, out string mensaje)
{
    mensaje = "";
    if (letra.Length != 1 || !char.IsLetter(letra[0]))
    {
        mensaje = "Ingresa una letra válida";
        return false;
    }
    if (!_letrasIngresadas.Contains(letra[0]))
    {
        mensaje = "Letra ya ingresada";
        return false;
    }
    return true;
}
```

Administración de Memoria

El sistema implementa buenas prácticas de gestión de memoria como el uso de `using`, liberación de recursos, y uso de colecciones eficientes.

```
public static Image ObtenerImagenRecurso(string nombreImagen)
{
    try
    {
        var assembly = Assembly.GetExecutingAssembly();

        string[] posiblesNombres = {
            $"Ahorcado.Resources.{nombreImagen}",
            $"Ahorcado.Resources.{Path.GetFileNameWithoutExtension(nombreImagen)}{Path.GetExtension(nombreImagen)}",
            $"Ahorcado.Resources.{Path.GetFileNameWithoutExtension(nombreImagen)}{Path.GetExtension(nombreImagen)}"
        };

        foreach (var resourceName in posiblesNombres)
        {
            using (var stream = assembly.GetManifestResourceStream(resourceName))
            {
                if (stream != null)
                {
                    return Image.FromStream(stream);
                }
            }
        }
    }
    catch { }
    return null;
}
```

El using garantiza que el stream se cierre y los recursos del sistema operativo se liberen. Además de evitar que los archivos de la imagen queden bloqueados en la memoria.

```
picAhorcado.Image = estado.ImagenAhorcado;
```

Al asignar una nueva imagen, el *runtime* libera la anterior.

```
private readonly HashSet<char> _letrasIncorrectas = new HashSet<char>();  
private readonly HashSet<char> _letrasIngresadas = new HashSet<char>();
```

El HashSet evita duplicados y al ser readonly, la referencia no puede reasignarse evita crear nuevas instancias accidentalmente.

```
_letrasIncorrectas.Clear();  
_letrasIngresadas.Clear();
```

Las letras almacenadas quedan elegibles para el *Garbage Collector* (GC).

Manejo de Errores

Implementación robusta de manejo de excepciones, validación de entradas, mensajes descriptivos y recuperación segura de recursos.

```
mensaje = "";  
if (letra.Length != 1 || !char.IsLetter(letra[0]))  
{
```

Previene entradas inválidas antes de procesar.

```
    catch (Exception ex)  
    {  
        Console.WriteLine($"Error al cargar imagen: {ex.Message}"); return null;  
    }
```

Programación Orientada a Objetos (POO)

El diseño sigue los principios de encapsulamiento, abstracción, herencia y polimorfismo con ejemplos claros en el código.

Abstracción

```
public ResultadoJuego ProcesarIntento(string letra)
{
    if (!ValidarLetra(letra, out var mensajeError))
        return new ResultadoJuego
        {
            Mensaje = mensajeError,
            Titulo = "Error",
            MostrarMensaje = true,
            NombreImagen = IMAGEN_ERROR
        };
}
```

Expone funcionalidad sin revelar implementación interna.

Encapsulación

```
private readonly List<string> palabras = new List<string>
{
    "guayaba", "arándano", "granadilla", "alcachofa", "berenjena", "armadillo", "ornitorrinco", "Neptuno", "Urano", "M
};
private List<string> pistas = new List<string>
{
    "Fruta tropical con muchas semillas", "Pequeña fruta morada", "Fruta tropical con semillas", "Verdura con muchas h
    "Animal semiacuático raro", "Planeta gaseoso azul", "Planeta con nombre de dios", "El más cercano al Sol", "Galaxi
};
```

Los campos están marcados como private para proteger el estado interno de modificaciones directas.

```
public class ResultadoJuego
{
    5 referencias
    public string Mensaje { get; set; }
    5 referencias
    public string Titulo { get; set; }
    5 referencias
    public string NombreImagen { get; set; }
    5 referencias
}
```

Permite controlar el acceso a los datos internos del objeto.

```
5 referencias  
public class ResultadoJuego  
{
```

5 referencias

Modela el resultado de una acción del juego como entidad independiente, ocultando los detalles de implementación.

Modularidad

```
4 referencias  
public class LogicaJuego  
{  
    private const string IMAGEN_ERROR = "Error.png";  
    private const string IMAGEN_CORRECTO = "Correcto.png";
```

Agrupar toda la lógica del juego en una unidad cohesiva.

Herencia

```
5 referencias  
public partial class Juego : Form  
{
```

Juego hereda de la clase Form de WinForms.

Programación Orientada a Eventos

El sistema responde a interacciones del usuario mediante eventos de botones, aplicando el patrón observer implícito del modelo de eventos de .NET.

```
private void btnEnviar_Click(object sender, EventArgs e)  
{  
    var resultado = _logica.ProcesarIntento(txtLetra.Text);  
  
    if (resultado.MostrarMensaje)  
        MostrarMensajePersonalizado(resultado.Mensaje, resultado.Titulo, resultado.NombreImagen);  
  
    if (resultado.ReiniciarJuego)  
    {
```

Flujo del Programa

1. Inicialización: Creación del Form principal y configuración de la interfaz.
2. Ciclo del juego: Entrada del usuario, validación, actualización de estado y verificación de resultado.
3. Finalización: Opción de nueva partida o confirmación de salida.

Consideraciones Técnicas

Recursos: Imágenes embebidas, reflexión para carga de recursos.

Persistencia: Estadísticas en memoria durante la sesión.

Internacionalización: Mensajes en español con posibilidad de multilenguaje.

Requisitos del Sistema

.NET Framework compatible con Windows Forms.

Sistema operativo Windows.

Resolución de pantalla mínima recomendada: 800x600.

Link de la aplicación

<https://github.com/Vanessa1356458/Ahorcado.git>

Código de la Lógica del Juego

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Reflection;
using System.Text;
using System.Threading.Tasks;
using System.Drawing;
using System.IO;
using System.Net.NetworkInformation;

namespace Ahorcado
{
    4 referencias
    public class LogicaJuego
    {
        private const string IMAGEN_ERROR = "Error.png";
        private const string IMAGEN_CORRECTO = "Correcto.png";
        private const string IMAGEN_GANASTE = "Ganaste.png";
        private const string IMAGEN_PERDISTE = "Perdiste.png";

        private readonly List<string> palabras = new List<string>
        {
            "guayaba", "arándano", "granadilla", "alcachofa", "berenjena", "armadillo", "ornitorrinco", "Neptuno", "Urano", "Mercurio", "Andrómeda", "Ganimedes"
        };
        private List<string> pistas = new List<string>
        {
            "Fruta tropical con muchas semillas", "Pequeña fruta morada", "Fruta tropical con semillas", "Verdura con muchas hojas", "Verdura morada", "Mamífero con caparazón",
            "Animal semiacuático raro", "Planeta gaseoso azul", "Planeta con nombre de dios", "El más cercano al Sol", "Galaxia vecina", "Luna más grande de Júpiter"
        };

        private readonly Random _random = new Random();

        private string _palabraActual;
        private string _pistaActual;
        private string _palabraOculto;
        private int _intentos = 6;
        private int _victorias = 0;
        private int _derrotas = 0;

        private readonly HashSet<char> _letrasIncorrectas = new HashSet<char>();
        private readonly HashSet<char> _letrasIngresadas = new HashSet<char>();

        2 referencias
        public void IniciarNuevoJuego()
        {
            int indiceAleatorio = _random.Next(palabras.Count);
            _palabraActual = palabras[indiceAleatorio];
            _pistaActual = pistas[indiceAleatorio];
            _palabraOculto = _palabraActual[0] + new string('_', _palabraActual.Length - 1); _intentos = 6;
            _letrasIncorrectas.Clear();
            _letrasIngresadas.Clear();
        }

        1 referencia
        public ResultadoJuego ProcesarIntento(string letra)
        {
            if (!ValidarLetra(letra, out var mensajeError))
                return new ResultadoJuego
                {
                    Mensaje = mensajeError,
                    Titulo = "Error",
                    MostrarMensaje = true,
                    NombreImagen = IMAGEN_ERROR
                };

            _letrasIngresadas.Add(letra[0]);

            if (_palabraActual.Contains(letra))
            {
                ActualizarPalabraOculto(letra[0]);
                if (_palabraOculto == _palabraActual)
                {
                    _victorias++;
                    return new ResultadoJuego
                    {
                        Mensaje = "¡Ganaste!",
                        Titulo = "Resultado",
                        MostrarMensaje = true,
                        ReiniciarJuego = true,
                        NombreImagen = IMAGEN_GANASTE
                    };
                }
            }

            return VerificarResultado("¡Correcto!", "Acierto", IMAGEN_CORRECTO);
        }
    }
}

```

```

    }
    else
    {
        _letrasIncorrectas.Add(letra[0]);
        _intentos--;
        if (_intentos <= 0)
        {
            _derrotas++;
            return new ResultadoJuego
            {
                Mensaje = $"Perdiste! La palabra era: {_palabraActual}",
                Titulo = "Resultado",
                MostrarMensaje = true,
                ReiniciarJuego = true,
                NombreImagen = IMAGEN_PERDISTE
            };
        }
        return VerificarResultado($"Incorrecto. Intentos: {_intentos}", "Error", IMAGEN_ERROR);
    }
}

```

1 referencia

```

public EstadoJuego ObtenerEstado()
{
    return new EstadoJuego
    {
        PalabraOculto = string.Join(" ", _palabraOculto.ToCharArray()),
        LetrasIncorrectas = "Letras incorrectas: " + string.Join(" ", _letrasIncorrectas),
        ImagenAhorcado = ObtenerImagenAhorcado(),
        Pista = "Pista: " + _pistaActual,
        Victorias = "Victorias: " + _victorias,
        Derrotas = "Derrotas: " + _derrotas
    };
}

```

1 referencia

```

private bool ValidarLetra(string letra, out string mensaje)
{
    mensaje = "";
    if (letra.Length != 1 || !char.IsLetter(letra[0]))
    {
        mensaje = "Ingresa una letra válida";
        return false;
    }
    if (_letrasIngresadas.Contains(letra[0]))
    {
        mensaje = "Letra ya ingresada";
        return false;
    }
    return true;
}

```

1 referencia

```

private void ActualizarPalabraOculto(char letra)
{
    var array = _palabraOculto.ToCharArray();
    for (int i = 0; i < _palabraActual.Length; i++)
        if (_palabraActual[i] == letra)
            array[i] = letra;
    _palabraOculto = new string(array);
}

```

2 referencias

```

private ResultadoJuego VerificarResultado(string mensaje, string titulo, string nombreImagen)
{
    return new ResultadoJuego
    {
        Mensaje = mensaje,
        Titulo = titulo,
        MostrarMensaje = true,
        NombreImagen = nombreImagen,
        ReiniciarJuego = false
    };
}

```

Conclusiones

El juego del Ahorcado está organizado para que sea de fácil de entender. Tiene dos partes principales: la pantalla que ven los jugadores y las reglas del juego, las cuales están separadas para que sea más sencillo hacer cambios. El programa funciona de manera estable y evita errores comunes.

Está hecho pensando en que se pueda mejorar después. Por ejemplo, sería fácil agregar más palabras o incluso nuevas formas de jugar. Todo está ordenado de manera lógica, lo que permite que cualquier persona que quiera modificarlo o agregarle cosas nuevas pueda hacerlo sin problemas.